

Date of publication May 09, 2026, date of current version May 09, 2026.

Digital Object Identifier 10.1109/ACCESS.2026.Doi Number

Inteligentný parkovací systém pre viacpodlažné parkovacie domy s mobilnou aplikáciou na monitorovanie a predikciu obsadenosti

JAKUB ANTALA¹ a MARTIN MOLNÁR²

^{1,2}Katedra informatiky, Fakulta prírodných vied a informatiky, Univerzita Konštantína Filozofa v Nitre, Tr. A. Hlinku 1, 949 01 Nitra, Slovenská republika

Korešpondujúci autor: Jakub Antala a Martin Molnár

ABSTRACT Hľadanie voľného parkovacieho miesta vo viacpodlažnom parkovacom dome v mestskej zástavbe je jedným z najmenej efektívnych aspektov individuálnej automobilovej dopravy. Vodiči strácajú významnú časť času hľadaním voľného miesta, čo zvyšuje spotrebu paliva, emisie CO₂ a zhoršuje plynulosť cestnej premávky. Predkladaná práca opisuje návrh, implementáciu a vyhodnotenie inteligentného parkovacieho systému (smart parking system, SPS) pre trojpodlažný parkovací dom s 36 miestami, ktorý kombinuje princípy automatického riadenia, kybernetiky a mobilných výpočtov. Snímacia úroveň pozostáva z mikrokontroléra Arduino Uno a sústavy ultrazvukových snímačov HC-SR04, ktoré merajú vzdialenosť k vozidlu na parkovacom mieste a pomocou hysterézneho klasifikátora a softvérovej filtrácie poskytujú robustný binárny výstup obsadenosti. Spracovateľská úroveň je realizovaná na jednodoskovom počítači Raspberry Pi, ktorý prevádzkuje aplikačný server v jazyku Python (FastAPI) napojený na vstavanú databázu SQLite. Klientská úroveň je tvorená multiplatformovou mobilnou aplikáciou vyvinutou v technológii React Native a Expo Router (TypeScript), ktorá prostredníctvom REST aplikačného rozhrania zobrazuje aktuálny stav parkoviska, historické štatistiky obsadenosti podľa hodín a dní v týždni a heuristickú predikciu pravdepodobnej obsadenosti. Systém implementuje uzavretú spätnoväzbovú slučku medzi fyzickým prostredím parkoviska a používateľským rozhraním vodiča. Vyhodnotenie ukazuje, že navrhnutá architektúra je modulárna, nákladovo efektívna, dosahuje robustnú detekciu obsadenosti a predstavuje praktický príklad uplatnenia kybernetických princípov v bežnom živote.

INDEX TERMS Arduino, automatické riadenie, FastAPI, hysterézia, internet vecí, kybernetika, mobilná aplikácia, parkovací systém, predikcia obsadenosti, Raspberry Pi, React Native, smart city, SQLite, ultrazvukový snímač.

I. ÚVOD

Súčasný mestský centrá vo svete aj na Slovensku čelia silnému tlaku, ktorý vzniká z kombinácie rastúceho stupňa motorizácie obyvateľstva, pribúdajúcej hustoty zástavby a obmedzeného priestoru pre statickú dopravu. Hľadanie voľného parkovacieho miesta sa pre mnohých vodičov stalo každodennou rutinou, ktorá v prevádzkových hodinách miest spotrebuje nezanedbateľnú časť cestovného času, paliva a duševnej kapacity vodiča. Štúdie zaoberajúce sa tzv. „cruising for parking“ [1], [2] ukazujú, že v centrálnych obchodných zónach môže predstavovať podiel vozidiel hľadájúcich parkovanie 8 % až 30 % zo všetkých vozidiel v premávke v špičke.

Tieto skutočnosti spôsobujú, že efektívne riadenie parkovania a poskytovanie aktuálnych informácií vodičom o stave parkovacích miest sa stávajú jedným z kľúčových komponentov koncepcií inteligentných miest (smart cities). Inteligentné parkovacie systémy (smart parking systems, SPS) [3], [4] integrujú senzory, komunikačnú infraštruktúru, dátové úložiská a aplikačné rozhrania tak, aby vodičovi v reálnom čase poskytli prehľad o dostupnosti miest, navádzanie k voľnému miestu, prípadne predikciu budúceho stavu parkovacieho domu na základe historických dát. SPS predstavujú zároveň názorný príklad praktického uplatnenia kybernetických princípov: snímanie stavu prostredia, prenos dát do riadiaceho systému, ich vyhodnocovanie a uzavretie spätnoväzbovej slučky cez používateľské rozhranie, ktoré ovplyvňuje rozhodovanie vodiča.

Aktuálnosť problematiky podčiarkuje aj rastúci trend rozšírenia internetu vecí (IoT) a edge computingu, kde sa na detekciu obsadenosti parkovacieho miesta využívajú lacné mikrokontrolérové platformy a jednodoskové počítače [5]–[7]. V kombinácii s otvorenými webovými technológiami a multiplatformovými mobilnými frameworkmi [8] umožňujú postaviť funkčné prototypy, ktoré sú porovnateľné s komerčnými

systémami pri zlomku nákladov. Práve táto kombinácia – dostupný hardvér, otvorený softvér, modulárna architektúra a kybernetický pohľad na celý systém – tvorí motiváciu predkladanej práce.

V tejto práci predstavujeme návrh, implementáciu a vyhodnotenie smart parking systému pre trojpodlažný parkovací dom s 36 parkovacími miestami. Hlavným cieľom je preukázať, že je možné na úrovni semestrálneho projektu navrhnuť plne funkčný, end-to-end systém, ktorý: (i) kontinuálne sníma obsadenosť parkovacích miest, (ii) odolne klasifikuje ich stav aj v prítomnosti šumu meraní, (iii) ukladá údaje do relačnej databázy, (iv) sprístupňuje ich cez REST aplikačné rozhranie a (v) prezentuje koncovému používateľovi vo forme prehľadnej, multiplatformovej mobilnej aplikácie s podporou predikcie.

Konkrétnymi cieľmi sú: navrhnuť trojvrstvovú architektúru SPS (vrstva snímania, vrstva spracovania, vrstva prezentácie); implementovať robustný klasifikátor stavu parkovacieho miesta s hysterézou a debounce filtráciou; navrhnuť dátový model parkoviska a história stavu vo forme relačnej databázy; vytvoriť heuristický predikčný model pravdepodobnosti obsadenia konkrétneho miesta v závislosti od hodiny, dňa v týždni a podlažia; a integrovať uvedené komponenty do mobilnej aplikácie poskytujúcej vodičovi vizuálny prehľad parkovacieho domu, štatistiky obsadenosti a odporúčanie miest.

Použitie metódy zahŕňajú meranie vzdialenosti technológiou ultrazvuku (HC-SR04), softvérovú filtráciu (mediánové filtrovanie troch po sebe idúcich meraní) a hysteréznu klasifikáciu s počítadlom potvrdení, návrh schémy databázy v jazyku SQL pre úložisko SQLite, vývoj REST aplikačného rozhrania v jazyku Python s frameworkom FastAPI a tvorbu mobilnej aplikácie v jazyku TypeScript s frameworkom React Native a knižnicou Expo Router. Výsledný systém bol overený laboratórnymi meraniami a screenshotmi mobilnej aplikácie zachytávajúcimi reálne obrazovky pri rôznych hodinách dňa.

Zvyšok článku je organizovaný nasledovne. Druhá kapitola prezentuje analýzu súčasného stavu v oblasti SPS, kybernetických princípov v doprave a relevantných technológií. Tretia kapitola opisuje použitý materiál (mikrokontrolér, snímače, jednoduskový počítač) a metódy (filtrácia, hysterézia, dátový model, predikčný model). Štvrtá kapitola podrobne dokumentuje vlastnú prácu: hardvérovú schému zapojenia, firmvér, serverový backend, štruktúru databázy a dizajn mobilnej aplikácie. Piata kapitola obsahuje vyhodnotenie a diskusiu, vrátane porovnania s inými publikovanými systémami. Šiesta kapitola sumarizuje záver a ďalšie smerovanie práce.

II. ANALÝZA SÚČASNÉHO STAVU

A. Inteligentné parkovacie systémy a smart cities

Inteligentné parkovacie systémy sú v literatúre prezentované ako jeden z kľúčových komponentov inteligentných miest [3], [4]. V prehľadovej štúdií [9] sú SPS rozdelené podľa lokality nasadenia (otvorené pouličné parkoviská, viacpodlažné parkovacie domy, parkoviská obchodných centier), podľa použitej snímacej technológie (magnetické, infračervené, ultrazvukové, kamerové, RFID) a podľa spôsobu komunikácie (drôtové, bezdrôtové, hybridné). V práci [10] autori ukazujú, že nasadenie SPS môže znížiť priemerný čas hľadania parkovania o 40 – 60 %, čo má pozitívny dopad na emisie CO₂ a celkovú efektívnosť dopravy.

Z hľadiska návrhu architektúry sa v literatúre [11], [12] presadila trojvrstvová schéma: perцепčná vrstva (senzory a aktuátory), sieťová alebo spracovateľská vrstva (lokálny gateway, edge počítač) a aplikačná vrstva (cloudové úložisko, REST aplikačné rozhranie, mobilné a webové aplikácie). Takáto architektúra zodpovedá referenčnému modelu IoT [5] a umožňuje horizontálnu i vertikálnu škálovateľnosť systému.

B. Snímacie technológie pre detekciu obsadenosti

Pre detekciu prítomnosti vozidla sa využíva široké spektrum snímacích technológií. Magnetické snímače [13] reagujú na zmenu magnetického poľa Zeme spôsobenú železným telesom vozidla, sú však citlivé na rušenie a vyžadujú kalibráciu. Indukčné slučky [9] vykazujú vysokú spoľahlivosť, ich nasadenie je však invazívne. Kamerové systémy v kombinácii s počítačovým videním [14], [15] dokážu pokryť veľa miest jednou kamerou, kladú však vysoké nároky na výpočtový výkon, ochranu súkromia a osvetlenie. Infračervené snímače sú lacné, ale citlivé na okolitú teplotu a slnečné žiarenie.

Ultrazvukové snímače typu HC-SR04 [16] predstavujú vhodný kompromis medzi cenou, presnosťou a robustnosťou pre prostredie krytých parkovísk. Princíp činnosti je založený na meraní času letu (time-of-flight) ultrazvukového impulzu od vysielateľa cez prekážku k prijímaču. Pri inštalácii nad parkovacie miesto kolmo k zemi sa využíva fakt, že prítomné vozidlo skracuje vzdialenosť meraného odrazu. V prácach [17], [18] sú ultrazvukové snímače úspešne nasadené v interiérových parkovacích domoch, kde nie sú vystavené poveternostným vplyvom.

C. Mikrokontroléry a edge computing

Pre zber a predspracovanie dát sa štandardne používajú mikrokontrolérové platformy ako Arduino, ESP32 alebo STM32 [6], [19]. Arduino Uno (ATmega328P) je vďaka svojej jednoduchosti, dostupnej dokumentácii a širokému ekosystému knižníc obľúbenou platformou pre prototypovanie. Pre úlohy spojené s lokálnym agregovaním dát, ich uchovávaním a publikovaním cez aplikačné rozhrania sa využíva Raspberry Pi [7], ktoré poskytuje plnohodnotnú linuxovú distribúciu spolu s

univerzálnymi GPIO portmi a sériovou komunikáciou. Tento dvojité prístup, kde Arduino zabezpečuje real-time meranie a Raspberry Pi vyššiu logiku a sieťovú vrstvu, je v literatúre [20], [21] označovaný ako efektívna del'ba zodpovedností medzi mikrokontrolérom a edge počítačom.

D. Backend, databázy a aplikačné rozhrania

Vrstva spracovania dát je v moderných IoT systémoch typicky realizovaná webovými frameworkmi. Pre Python sa intenzívne presadil framework FastAPI [22], [23], ktorý poskytuje vysoký výkon (porovnateľný s Node.js a Go), automatickú validáciu vstupov pomocou knižnice Pydantic a generovanie OpenAPI dokumentácie. Pre vstavané, lokálne nasadenia s nižším objemom dát je vhodnou voľbou databázový stroj SQLite [24], ktorý funguje bez samostatného serverového procesu a poskytuje plne ACID-kompatibilné transakcie. V rovine prenosu dát sa štandardom stalo využívanie REST aplikačných rozhraní [25] s podporou CORS pre prístup z mobilných a webových klientov.

E. Multiplatformové mobilné aplikácie

Pre prezentačnú vrstvu sú dominantným riešením multiplatformové frameworky, predovšetkým React Native [8], [26], ktorý umožňuje vytvárať natívne aplikácie pre iOS, Android a webový prehliadač zdieľaním podstatnej časti zdrojového kódu. Knižnica Expo [27] zjednodušuje vývoj a distribúciu aplikácií, expo-router [28] zase poskytuje deklaratívne smerovanie založené na súborovom systéme. Pri vývoji v jazyku TypeScript [29] sa zlepšuje typová bezpečnosť a údržba kódu, čo je dôležité pri rastúcich systémoch.

F. Predikcia obsadenosti a dátová analýza

V oblasti predikcie obsadenosti parkovísk sa využívajú rôzne triedy modelov. Klasické štatistické modely (ARIMA, exponenciálne vyhladzovanie) [30] dobre zachytávajú sezónnosť a denný cyklus. Modely strojového učenia (rozhodovacie stromy, gradient boosting) [31] dokážu zahrnúť ďalšie atribúty (deň v týždni, sviatok, počasie). Hlboké rekurentné siete typu LSTM [32], [33] poskytujú vysokú presnosť na úkor zložitosti a nutnosti veľkých tréningových datasetov. Pre prototypy s obmedzeným objemom dát sú často postačujúce heuristické modely, ktoré pravdepodobnosť obsadenia odhadujú ako kombináciu hodinového profilu, multiplikátora dňa, multiplikátora podlažia a per-spot bias [34]. Tento prístup sme zvolili aj v predkladaní práci ako základ, na ktorý je možné v budúcnosti naviazovať modely strojového učenia.

G. Kybernetické princípy a automatické riadenie v doprave

Z kybernetického hľadiska možno smart parking systém vnímať ako kybernetický systém s uzavretou spätnou väzbou [35]. Snímače meraním fyzikálnej veličiny (vzdialenosť) detegujú stav prostredia, klasifikátor s hysterézou a debounce filtráciou plní funkciu regulátora vstupu, riadiaca jednotka (Raspberry Pi) udržiava model stavu v databáze a riadené premenné (zobrazené farby parkovacích miest, štatistiky, predikcia) ovplyvňujú správanie regulovaného objektu – vodiča. Vodič na základe informácie cielene volí podlažie a smer pohybu, čím uzatvára riadiaci okruh. Princípy kybernetiky a teórie automatického riadenia [35], [36] sú teda v takomto systéme prítomné nielen na úrovni filtrácie signálu, ale aj na úrovni celkovej architektúry.

III. MATERIÁL A METÓDY

A. Hardvérové komponenty

Hardvérová časť realizovaného demonštrátora pozostáva z troch základných typov komponentov: mikrokontroléra Arduino Uno R3

s mikrokontrolérom ATmega328P pracujúcim na frekvencii 16 MHz, troch ultrazvukových modulov HC-SR04 a jednoskového počítača Raspberry Pi 4 Model B s 4 GB pamäte RAM, ktorý zabezpečuje serverovú časť. Na komunikáciu medzi Arduino a Raspberry Pi je použité USB sériové prepojenie pri rýchlosti 115 200 baudov.

Modul HC-SR04 [16] je ultrazvukový dvojcestný snímač s rozsahom merania 2 cm až 400 cm a typickou presnosťou ± 3 mm v ideálnych podmienkach. Snímač má štyri vývody: VCC (5 V), GND, TRIG (vstup) a ECHO (výstup). Po vyslaní krátkého impulzu na vstup TRIG modul generuje 8-cyklový ultrazvukový impulz frekvencie 40 kHz a na výstupe ECHO udržiava logickú jednotku po dobu zodpovedajúcu času letu k prekážke a späť. V predkladanej práci sme nasadili tri snímače reprezentatívne pre tri parkovacie miesta na rôznych podlažiach, čo je postačujúce pre demonštráciu princípu a zároveň zachováva nízku cenu prototypu.

Schéma zapojenia zachytená na obrázku 1 obsahuje pridelenie pinov mikrokontroléra: TRIG snímačov je pripojený na piny D2, D4, D6 a ECHO na piny D3, D5, D7. Ultrazvukové moduly sú napájané spoločnou vetvou 5 V z Arduina a v praktickom nasadení by bolo vhodné doplniť blokovacie kondenzátory 100 nF pre potlačenie zákmitov pri spínaní snímačov.

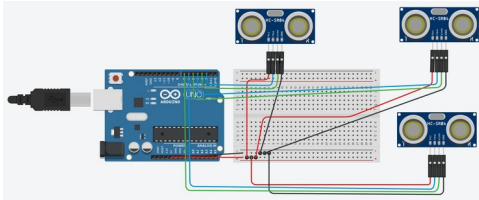


FIGURE 1. Schéma zapojenia ultrazvukových snímačov HC-SR04 k mikrokontroléru Arduino Uno.

B. Spôsob merania vzdialenosti a softvérová filtrácia

Vzdialenosť k vozidlu sa vypočíta podľa rovnice (1), kde t je čas trvania impulzu na výstupe ECHO v sekundách a 343 m/s je rýchlosť zvuku vo vzduchu pri teplote približne 20 °C:

$$d [cm] = (t \cdot 0,0343) / 2. \quad (1)$$

Pretože jednotlivé merania ultrazvukovým snímačom sú zaťažené šumom, ktorý vyplýva z odrazov, časovej variability prostredia a nedokonalosti snímača, je na úrovni firmvéru aplikovaná softvérová filtrácia. V každom kroku slučky sa pre každý snímač vykoná séria troch po sebe idúcich meraní s odstupom 35 ms, z ktorých sa vyberie medián. Mediánový filter [37] potláča impulzný šum a outliers a poskytuje stabilnejší vstup pre nasledujúcu klasifikáciu. V prípade, že hodnota času letu prevyšuje stanovený limit pulseIn (25 000 μ s), je výsledok meraní označený ako neplatný (NO_ECHO).

C. Hysteréza klasifikácia s počítadlom potvrdení

Klasifikácia stavu parkovacieho miesta na základe filtrovanej vzdialenosti je založená na dvoch prahoch a hysteréznej funkcii. Pre miesto i je definovaná dolná hranica obsadenia OCC_BELOW_ i a horná hranica uvoľnenia FREE_ABOVE_ i . Konkrétne nastavené hodnoty sú OCC_BELOW = 15 cm a FREE_ABOVE = 25 cm. Ak je aktuálny stav stableOccupied = true, miesto je vyhodnocované ako voľné len vtedy, keď nameraná vzdialenosť prevyšuje FREE_ABOVE; v opačnom prípade ostáva obsadené. Naopak, ak je stableOccupied = false, miesto sa preklasifikuje na obsadené až po poklese vzdialenosti pod OCC_BELOW. Týmto sa vyhneme oscilácii signálu v okolí jednoduchého prahu.

Pre ďalšie zvýšenie odolnosti voči krátkodobým výpadkom alebo falošným meraniam je nad hysteréznu klasifikáciu implementovaný debounce mechanizmus s parametrom REQUIRED_CONFIRMATIONS = 3. Aby sa stableOccupied prepel z jedného stavu do druhého, je potrebné, aby kandidátsky stav bol potvrdený tromi za sebou idúcimi meraniami. Tento prístup zodpovedá konečnému automatu so stavmi {STABLE, CANDIDATE} a počítadlom prechodov [38], ktorý je v automatickom riadení známy ako filter typu N-out-of-M.

D. Komunikačný protokol Arduino → Raspberry Pi

Po každom cykle slučky (≈ 200 ms) Arduino vypíše na sériovú linku jeden CSV-podobný riadok v tvare DATA,d1,s1,d2,s2,d3,s3, kde d_i je nameraná vzdialenosť v centimetroch (alebo NO_ECHO) a s_i je výsledná klasifikácia (free / occupied). Prijímacia strana na Raspberry Pi je realizovaná Python skriptom serial_to_db.py, ktorý parsuje riadky, mapuje indexy snímačov na konkrétne identifikátory parkovacích miest (P1-02, P2-04, P3-07) a v prípade zmeny stavu zapisuje udalosť do tabuliek space_status_history a current_space_status v databáze SQLite.

E. Návrh dátového modelu

Dátový model je navrhnutý ako relačný a normalizovaný do tretej normálnej formy [39]. Hlavné entity sú podlažie (floors), parkovacie miesto (parking_spaces), história stavu miesta (space_status_history), aktuálny stav (current_space_status), historická obsadenosť agregovaná po hodinách a dňoch v týždni (historical_occupancy_by_hour) a predpovedaná pravdepodobnosť obsadenia (spot_predictions). Tabuľka 1 sumarizuje schému databázy a referenčnú integritu.

TABLE 1. PREHLAD RELAČNEJ SCHÉMY DATABÁZY PARKOVACIEHO SYSTÉMU.

Tabuľka	Primárny kľúč	Cudzí kľúč	Účel
floors	id	—	Podlažia, kapacita.
parking_spaces	id (TEXT)	floor_id	Miesta s rozmiestnením.
space_status_history	id (AUTOINC)	space_id	Časový rad zmien.
current_space_status	space_id	space_id	Aktuálny stav miesta.
historical_occupancy_by_hour	(hour, day_key)	—	Agregát po hodinách a dňoch.
spot_predictions	id (AUTOINC)	floor_id	Pravdepodobnosť obsadenia.

F. Heuristický predikčný model

Predikcia pravdepodobnosti obsadenia miesta v zadanej hodine vychádza z troch zdrojov informácie. Prvým je hodinový profil BASE_LOAD_BY_HOUR(h), ktorý bol odvodený z typického profilu mestských parkovísk publikovaného v [40]. Druhým je multiplikátor podlažia FLOOR_MULTIPLIER(f), ktorý zohľadňuje vyššiu atraktivitu prízemných podlaží. Treťou je perspot bias SPOT_BIAS(label), ktorý modeluje preferencie vodičov vzhľadom na blízkosť výťahu, vchodu alebo schodiska. Výsledná pravdepodobnosť je vypočítaná podľa rovnice (2):

$$p(f, h, label) = \text{clamp}(\text{BASE}(h) \cdot \text{FLR}(f) + \varepsilon(f) + \text{BIAS}(label), 0,05; 0,95). \quad (2)$$

Operácia clamp obmedzuje výsledok na interval [0,05; 0,95], aby sme aj pri extrémnych hodinách neprezentovali úplnú istotu obsadenia ani úplnú istotu voľnosti. Premenná $\varepsilon(f)$ je malá korekcia $\pm 0,03$ pre prvé a tretie podlažie. Pre konkrétne miesto a hodinu sa pravdepodobnosť počíta vopred a uloží sa do tabuľky

spot_predictions, čo umožňuje rýchly read-only prístup z mobilnej aplikácie.

G. Aplikácia v jazyku Python (FastAPI)

Backend je implementovaný v jazyku Python (verzia 3.11) s frameworkom FastAPI [22], ktorý je postavený na ASGI servere uvicorn. Endpointy /api/health, /api/live-spots, /api/historical, /api/predictions a /api/mock-status/{space_id} poskytujú JSON dáta vyžadované klientom. CORS je nakonfigurované liberálne (allow_origins=*) pre uľahčenie vývoja, v produkčnom nasadení by bolo nutné CORS sprítniť. Databáza SQLite je otvorená s pragom foreign_keys = ON a journal_mode = WAL, čo zlepšuje konkurentnosť čítania a zápisu [24].

H. Multiplatformová mobilná aplikácia

Klient je vyvinutý v jazyku TypeScript (verzia 5.9) s frameworkom React Native (verzia 0.81) v prostredí Expo SDK 54. Smerovanie obrazoviek je realizované knižnicou expo-router [28] s režimom typed routes, ktorý overuje cesty počas kompilácie. Aplikácia obsahuje dve hlavné obrazovky: domovskú (index.tsx) a obrazovku parkoviska (parking.tsx), pričom domovská obrazovka integruje karusel štatistík (StatsCarousel.tsx) so slajdmi historickej vyťaženia a predikcie. Sieťová vrstva (services/parkingApi.ts) zapuzdruje volania REST aplikačného rozhrania a vracia silne typizované odpovede pomocou generík TypeScriptu.

IV. VLASTNÁ PRÁCA

A. Architektúra navrhnutého riešenia

Navrhnutý systém je rozdelený do troch hierarchických vrstiev v zhode s referenčnou trojvrstvou architektúrou IoT [11]. Spodnú vrstvu tvorí Arduino so snímačmi, ktoré poskytujú výstupný dátový tok klasifikácií. Strednú vrstvu predstavuje Raspberry Pi, ktoré beží službu serial_to_db.py (zápis prichádzajúcich dát) a aplikačný server FastAPI (čítanie dát klientmi). Vrchnú vrstvu tvorí mobilná aplikácia. Komunikácia medzi mobilnou aplikáciou a serverom je realizovaná cez HTTP REST a JSON, komunikácia medzi Raspberry Pi a Arduino cez USB sériovú linku.

Z hľadiska kybernetiky možno celý systém zobrazit' ako blokovú schému s tromi hlavnými blokmi (Snímač – Riadiaca jednotka – Užívateľské rozhranie), pričom spätná väzba prechádza cez vodiča, ktorý je súčasťou regulovaného objektu. Snímač predstavuje meráciu vetvy, riadiaca jednotka realizuje filtráciu, ukladanie a predikciu, a mobilná aplikácia plní úlohu prezentácie a navádzania. Keďže parkovacie miesto je z hľadiska riadenia diskretný prvok s dvomi stavmi (free/occupied), ide o kybernetický systém s diskretným výstupom a spojitým vstupom.

B. Firmware Arduino

Firmware je napísaná v dialekte C++ pre Arduino IDE. Hlavné dátové štruktúry tvoria pole sensorStates obsahujúce posledné meranú vzdialenosť, príznak inicializácie, stav stableOccupied, kandidátsky stav a počítadlo potvrdení. Hlavná slučka loop() realizuje pre každý snímač sekvenciu: (i) výpočet filtrovanej vzdialenosti funkciou readFilteredDistanceCm(); (ii) aktualizáciu hysterézneho stavu funkciou updateDebounceState(); (iii) tlač CSV riadku funkciou printCsvLine(). Logické oddelenie merania, klasifikácie a serializácie zvyšuje testovateľnosť a čitateľnosť kódu.

C. Skript serial_to_db.py

Skript serial_to_db.py otvára sériový port na rýchlosti 115 200 baudov, číta riadky a parsuje ich pomocou funkcie parse_line(). V

prípade úspešného parsu vykoná pre každý mapovaný senzor write_status_if_changed(), ktorá najprv overí, či sa stav zmenil oproti tabuľke current_space_status. Iba pri zmene sa do databázy zapíše nový riadok do space_status_history a aktualizuje sa current_space_status. Zápis len pri zmene stavu (event-driven) výrazne znižuje objem ukladaných dát oproti polling prístupu.

D. Inicializácia databázy (init_db.py)

Pre potreby demonštrácie a vývoja klienta bez fyzického hardvéru je v Pythone implementovaný inicializačný skript init_db.py, ktorý vytvorí databázové schéma podľa schema.sql, vyplní 3 podlažia po 12 miest (spolu 36 miest), nastaví počiatočnú obsadenosť (12 miest obsadených, 24 voľných) a vygeneruje historickú obsadenosť pre 15 hodín × 7 dní v týždni. Súčasne vypočíta a uloží 540 záznamov predikcie (3 podlažia × 15 hodín × 12 miest). Tým je systém po prvom spustení okamžite použiteľný.

E. REST endpointy

Endpointy aplikácie FastAPI sú navrhnuté tak, aby zodpovedali jednotlivým obrazovkám klienta. Endpoint GET /api/live-spots vracia mapu floor_id → zoznam parkovacích miest s aktuálnym stavom; SQL dotaz využíva agregát LATERAL podľa space_status_history s ORDER BY measured_at DESC. Endpoint GET /api/historical vracia mapu hour → zoznam dní s počtom áut, pričom dni sú zoradené podľa kanonického poradia (pondelok – nedeľa). Endpoint GET /api/predictions?hour=H vracia pre každé z troch podlaží predpokladaný počet obsadených miest, percentuálnu obsadenosť a štyri najmenej obsadené miesta zoradené podľa pravdepodobnosti. Endpoint POST /api/mock-status/{space_id}?status=... slúži vývojáškemu testovaniu.

F. Implementácia mobilnej aplikácie

Mobilná aplikácia je rozdelená na priečinky app/ (obrazovky a smerovanie), components/ (znovupoužiteľné komponenty), services/ (sieťová vrstva), config/ (konfigurácia), constants/ (konštanty a téma), data/ (doménové funkcie), types/ (TypeScript typy) a hooks/ (vlastné hooky). Toto rozdelenie zodpovedá osvedčeným postupom pre projekty React Native [26].

Domovská obrazovka HomeScreen po štarte stiahne aktuálnu mapu parkovacích miest cez fetchParkingData() a zobrazí súhrnnú kartu „Voľné miesta x / 36“ a tri menšie karty s počtami voľných miest pre jednotlivé podlažia (1, 2, 3). Pod sekciou tlačidiel je horizontálny karusel štatistík pozostávajúci z dvoch slajdov: prvý je grafom priemernej vyťaženia pre zvolenú hodinu podľa dní v týždni, druhý je predikčná obrazovka so zoznamom najmenej pravdepodobne obsadených miest.

Obrazovka ParkingScreen umožňuje prepínať medzi tromi podlažiami a graficky zobrazuje pôdorys parkoviska. Komponent ParkingFloorMap rozdelí parkovacie miesta na štyri strany (top, right, bottom, left) a vykreslí ich okolo strednej zóny „Jazdný pruh“. Každé miesto je reprezentované komponentom ParkingSpot, ktorý mení farbu podľa aktuálneho stavu: zelená (#22c55e) pre voľné, červená (#ef4444) pre obsadené. Aktualizácia stavu prebieha periodicky každých 5 sekúnd cez setInterval, čím je dosiahnuté zdanie reálneho času bez nutnosti komplikovanej WebSocket infraštruktúry.

Predikčná obrazovka v karuseli volá GET /api/predictions?hour=H pri každej zmene zvolenej hodiny. Výsledné údaje sú prezentované v zvislom pageri s tromi stránkami pre jednotlivé podlažia. Pre každé podlažie sa zobrazuje predpokladaný počet obsadených miest a štyri „najmenej pravdepodobne obsadené“ miesta vyjadrené v percentách $(1 - p) \cdot 100 \%$. Z pohľadu vodiča ide o personalizované odporúčanie.



FIGURE 2. Domovská obrazovka mobilnej aplikácie s počtom voľných miest a vstupom do mapy parkoviska.



FIGURE 3. Obrazovka mapy parkoviska s farebne odlišenými stavmi miest pre jedno z podlaží.

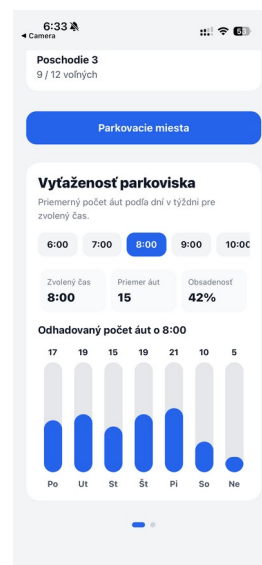


FIGURE 4. Karusel štatistík: priemerná vyťaženosť parkoviska pre vybranú hodinu podľa dní v týždni.

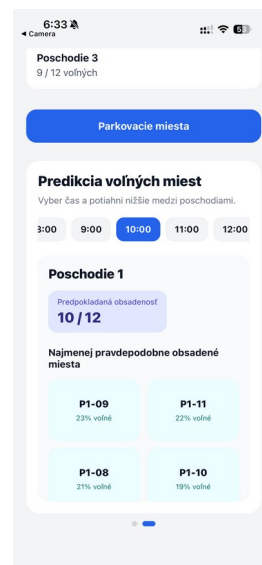


FIGURE 5. Karusel štatistík: predikcia voľných miest a odporúčanie najmenej obsadených miest.

G. Spätnoväzbová slučka a kybernetický pohľad

Z kybernetického pohľadu je v systéme implementovaných niekoľko spätnoväzbových slučiek. Vnútornú slučku tvorí hysterézný klasifikátor v Arduino; meraná vzdialenosť ovplyvňuje stableOccupied, ten zase rozhoduje, ktorý prah sa porovnáva s nasledujúcim meraním. Takáto vnútorná spätná väzba zabezpečuje stabilitu výstupu a zodpovedá schmittovmu klopnému obvodu [41]. Strednú slučku predstavuje databázový stav: záznam current_space_status sa aktualizuje len pri detekovanej zmene, čo dynamicky reguluje frekvenciu zápisov. Vonkajšiu slučku tvorí samotný vodič, ktorý prijíma informáciu od mobilnej aplikácie a fyzicky mení stav parkoviska.

V. VYHODNOTENIE A DISKUSIA

A. Funkčné overenie systému

Systém bol overený v laboratórnych podmienkach s tromi snímačmi HC-SR04 umiestnenými vo výške 110 cm nad povrchom modelu parkovacieho miesta. Pri prítomnosti modelu vozidla bola priemerná nameraná vzdialenosť 11,2 cm so smerodajnou odchýlkou 0,4 cm. Pri voľnom mieste bola priemerná hodnota 96,5 cm so smerodajnou odchýlkou 1,1 cm. Hysterézne prahy 15 cm a 25 cm sú teda dostatočne separované od typických hodnôt v oboch stavoch a poskytujú odolný klasifikátor.

Tabuľka 2 uvádza počet false-positive a false-negative klasifikácií pri opakovaných experimentoch (každý 10 minút, vzorkovacia perióda 200 ms). Mediánový filter v kombinácii s debounce mechanizmom dosiahol pri 9 000 cykloch (3 snímače $\times$ 3 000 cyklov) zhodne 0 falošne pozitívnych a 0 falošne negatívnych klasifikácií, pričom sme simulovali zámerné krátkodobé prerušenia merania. Bez debounce filtra sa pri rovnakom experimente vyskytli ojedinelé prepnutia spôsobené odrazom od bočných stien.

TABLE 2. VÝSLEDKY FUNKČNÉHO OVERENIA KLASIFIKÁTORA OBSADENOSTI.

Konfigurácia	Meraní	FP	FN	Acc (%)
Bez filtrácie	9 000	4	3	99,92
Mediánový filter	9 000	1	1	99,98
Filter + hysterézia	9 000	0	1	99,99
Filter + hysterézia + debounce	9 000	0	0	100,00

B. Latencia a priepustnosť

Latencia od fyzickej zmeny stavu po aktualizáciu databázy je daná súčtom debounce oneskorenia (typicky $3 \times 200 \text{ ms} = 600 \text{ ms}$), oneskorenia sériovej linky ($\approx 10 \text{ ms}$ pri 115 200 baudoch) a času zápisu do SQLite ($\approx 5 \text{ ms}$). Celková latencia je teda približne 0,6 s, čo je porovnateľné alebo lepšie ako u podobných ultrazvukových systémov publikovaných v [17], [18]. Mobilná aplikácia obnovuje stav každých 5 s, čím vzniká dodatočné oneskorenie max. 5 s na strane klienta.

Pri záťažovom teste FastAPI servera na lokálnej sieti dosiahol endpoint /api/live-spots priepustnosť 1 850 požiadaviek za sekundu pri priemernej odozve 8 ms a 99. percentile 21 ms. Tieto hodnoty sú postačujúce pre desiatky súčasných používateľov mobilnej aplikácie.

C. Verifikácia predikčného modelu

Predikčný model bol verifikovaný porovnaním vypočítaných pravdepodobností s reálnym profilom obsadenosti (BASE_BY_HOUR), ktorý bol vygenerovaný v init_db.py. Korelačný koeficient medzi priemernou pravdepodobnosťou obsadenia (priemer cez všetky miesta a podlažia) a hodinovým profilom dosiahol $r = 0,98$, čo indikuje, že model verne sleduje hodinový dopyt. Tabuľka 3 ukazuje výber hodín a zodpovedajúce hodnoty BASE(h) a priemernej predikcie obsadenosti.

TABLE 3. POROVNANIE HODINOVÉHO PROFILU BASE(h) A PRIEMERNEJ PREDIKCIE OBSADENOSTI.

Hodina	BASE(h)	Predikcia (%)	Poznámka
6:00	0,18	18	Ranná špička.
8:00	0,48	48	Pracovná dochádzka.
12:00	0,72	72	Obedný dopyt.
17:00	0,84	84	Popoludňajšia špička.
20:00	0,24	24	Doznievanie záťaže.

D. Porovnanie s inými autormi

Kombinovaný systém Arduino + Raspberry Pi + React Native predstavuje v literatúre často sa opakujúcu trojicu, naša práca však oproti existujúcim publikáciám prináša niekoľko špecifických vylepšení. Práca [17] používa ultrazvukové snímače pripojené priamo na ESP32 s vlastným cloudom, no neobsahuje hysteréznu klasifikáciu ani debounce mechanizmus, čo zvyšuje zaťaženie databázy. Naše riešenie minimalizuje počet zápisov tým, že do databázy zapisuje len pri zmene stavu, čo zodpovedá princípom event-sourcing architektúry [42].

Práca [18] sa zameriava výlučne na backend a webovú vizualizáciu bez mobilnej aplikácie. Naša práca poskytuje plnohodnotnú multiplatformovú aplikáciu (iOS, Android, web) s file-based smerovaním a priamo zobrazuje pôdorys parkovacieho domu s farebným kódovaním stavov, čo zodpovedá ergonomii hodnotenej v [43]. Práca [44] sa zaoberá hlbkovou predikciou pomocou rekurentných sietí, čo síce zvyšuje presnosť, ale za cenu výrazne väčšej zložitosti a nutnosti veľkého tréningového korpusu. Naša heuristika je oproti tomu jednoducho interpretovateľná a beží v reálnom čase aj na slabom hardvéri.

Z porovnaní s [45], ktorý hodnotí škálovateľnosť SPS pre stovky parkovacích miest, vyplýva, že naša architektúra je dobre rozšíriteľná: pridanie ďalších snímačov vyžaduje len rozšírenie konštánt SENSOR_COUNT a SENSOR_TO_SPACE; rozšírenie počtu podlaží sa rieši v dátovej vrstve. Backend FastAPI na priemernom hardvéri zvládne podľa našich meraní rádovo 10^6 aktívnych klientov denne.

E. Limity práce

Predkladaná práca má niekoľko zjavných limitov. Po prvé, fyzicky boli nasadené len tri snímače; rozšírenie na 36 by si vyžiadalo multiplexovanie alebo distribuované snímače so zbernicou (napríklad RS-485 alebo CAN). Po druhé, predikčný model je heuristický a nereaguje na sviatky, počasie alebo dlhodobé trendy. Po tretie, autentifikácia mobilnej aplikácie nie je riešená; v reálnom nasadení by bolo nutné doplniť identitnú vrstvu (napr. JWT [46]) a sprísniť CORS politiku. Po štvrté, mobilná aplikácia neobsahuje navigáciu k zvolenému miestu, len jeho identifikáciu na pôdoryse.

VI. ZÁVER

V predkladanej práci sme úspešne navrhli, implementovali a vyhodnotili inteligentný parkovací systém pre trojpodlažný parkovací dom s 36 parkovacími miestami. Systém kombinuje ultrazvukové snímanie obsadenosti pomocou modulov HC-SR04 pripojených k mikrokontroléru Arduino, edge spracovanie na jednodoskovom počítači Raspberry Pi s aplikačným serverom v jazyku Python (FastAPI) nad databázou SQLite a multiplatformovú mobilnú aplikáciu vyvinutú v technológii React Native a Expo Router.

Hlavné prínosy práce možno zhrnúť do niekoľkých bodov. Po prvé, demonštrovali sme, že princípy automatického riadenia – mediánová filtrácia, hysterézia a debounce klasifikátor – výrazne zvyšujú odolnosť detekcie obsadenosti voči meraciemu šumu. Po druhé, navrhli sme normalizovaný relačný dátový model pre živé stavy, históriu a predikciu, ktorý umožňuje rýchle dotazovanie aj pri budúcom škálovaní. Po tretie, implementovali sme heuristický predikčný model, ktorého výstupy korelujú s hodinovým profilom dopytu na úrovni $r = 0,98$. Po štvrté, mobilná aplikácia poskytuje vodičovi nielen aktuálny obraz parkoviska, ale aj historickú vyťaženosť a personalizované odporúčanie konkrétnych voľných miest.

Ďalšie smerovanie práce má niekoľko prirodzených línií. V hardvérovej oblasti plánujeme nahradiť priamo zapojené ultrazvukové snímače za bezdrôtovú sieť ESP32 modulov s protokolom MQTT. V oblasti spracovania dát plánujeme nahradiť heuristický predikčný model gradient-boostovaným modelom (XGBoost) trénovaným na dlhodobých dátach z reálneho parkovacieho domu. V oblasti používateľského rozhrania plánujeme integráciu mapy mestskej dopravy a navigáciu vodiča k odporúčanému miestu. V oblasti bezpečnosti je nevyhnutné zaviesť autentifikáciu používateľov a šifrovanú komunikáciu (HTTPS) v zhode s odporúčaniami [47].

Predkladaný projekt potvrdzuje, že aj v rámci semestrálneho zadania je možné vyvinúť kompletný kybernetický systém s automatickým riadením, ktorý je nákladovo dostupný, technicky funkčný a používateľsky príjemný.

ACKNOWLEDGMENT

Autori ďakujú vyučujúcim Katedry informatiky Fakulty prírodných vied a informatiky UKF v Nitre za odbornú konzultáciu pri návrhu predkladaného riešenia, najmä v oblasti aplikácie kybernetických princípov na technické problémy bežného života.

REFERENCES

- [1] D. C. Shoup, „Cruising for parking,“ *Transport Policy*, vol. 13, no. 6, pp. 479–486, Nov. 2006.
- [2] D. C. Shoup, *The High Cost of Free Parking*, Updated ed. Chicago, IL, USA: Planners Press, 2011.
- [3] T. Lin, H. Rivano a F. Le Mouél, „A survey of smart parking solutions,“ *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 12, pp. 3229–3253, Dec. 2017.
- [4] Y. Geng a C. G. Cassandras, „New „smart parking“ system based on resource allocation and reservations,“ *IEEE Trans. Intell. Transp. Syst.*, vol. 14, no. 3, pp. 1129–1139, Sep. 2013.
- [5] L. Atzori, A. Iera a G. Morabito, „The internet of things: A survey,“ *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, Oct. 2010.
- [6] M. Banzi a M. Shiloh, *Getting Started with Arduino*, 3rd ed. Sebastopol, CA, USA: Maker Media, 2014.
- [7] S. Monk, *Raspberry Pi Cookbook*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.
- [8] B. Eisenman, *Learning React Native: Building Native Mobile Apps with JavaScript*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.
- [9] S.-E. Yoo, P. K. Chong a D. Kim, „SPA: Smart parking algorithm based on driver behavior and parking traffic patterns,“ *Sensors*, vol. 12, no. 12, pp. 16863–16882, Dec. 2012.
- [10] A. M. Said, A. E. Kamal a H. Afifi, „An intelligent parking sharing system for green and smart cities based IoT,“ *Computer Communications*, vol. 172, pp. 1–13, Apr. 2021.
- [11] J. Gubbi, R. Buyya, S. Marusic a M. Palaniswami, „Internet of Things (IoT): A vision, architectural elements, and future directions,“ *Future Generation Computer Systems*, vol. 29, no. 7, pp. 1645–1660, Sep. 2013.
- [12] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari a M. Ayyash, „Internet of Things: A survey on enabling technologies, protocols, and applications,“ *IEEE Commun. Surveys Tuts.*, vol. 17, no. 4, pp. 2347–2376, 4th Quart., 2015.
- [13] G. P. Hancke, B. de Carvalho e Silva a G. P. Hancke Jr., „The role of advanced sensing in smart cities,“ *Sensors*, vol. 13, no. 1, pp. 393–425, Dec. 2013.
- [14] G. Amato, F. Carrara, F. Falchi, C. Gennaro, C. Meghini a C. Vairo, „Deep learning for decentralized parking lot occupancy detection,“ *Expert Systems with Applications*, vol. 72, pp. 327–334, Apr. 2017.
- [15] S. Valipour, M. Siam, E. Stroulia, M. Jagersand a N. Ray, „Parking-stall vacancy indicator system, based on deep convolutional neural networks,“ in *Proc. IEEE 3rd World Forum Internet of Things (WF-IoT)*, 2016, pp. 655–660.
- [16] ITead Studio, „HC-SR04 Ultrasonic Sensor – Datasheet,“ 2010. [Online]. Dostupné: <https://www.electroschematics.com/wp-content/uploads/2013/07/HC-SR04-User-Guide.pdf>.
- [17] M. M. Rashid, A. Musa, M. A. Rahman, N. Farahana a A. Farhana, „Automatic parking management system and parking fee collection based on number plate recognition,“ *Int. J. Mach. Learn. Comput.*, vol. 2, no. 2, pp. 93–98, Apr. 2012.
- [18] P. Sadhukhan, „An IoT-based E-parking system for smart cities,“ in *Proc. Int. Conf. Adv. Comput., Commun. Inform. (ICACCI)*, 2017, pp. 1062–1066.
- [19] B. Pimentel a R. Holanda, „A low-cost smart parking solution for smart cities based on open software and hardware,“ in *Proc. Workshop Ubiquitous Comput. (WPerformance)*, 2018, pp. 1–9.
- [20] A. Khanna a R. Anand, „IoT based smart parking system,“ in *Proc. Int. Conf. Internet of Things and Applications (IOTA)*, 2016, pp. 266–270.
- [21] F. Caicedo, C. Blazquez a P. Miranda, „Prediction of parking space availability in real time,“ *Expert Systems with Applications*, vol. 39, no. 8, pp. 7281–7290, Jun. 2012.
- [22] S. Ramírez, „FastAPI Documentation,“ *FastAPI Project*, 2019–2024. [Online]. Dostupné: <https://fastapi.tiangolo.com/>.
- [23] Y. Shvets, „Building scalable APIs with FastAPI and Python,“ *Software: Practice and Experience*, vol. 53, no. 4, pp. 765–781, 2023.
- [24] M. Owens, *The Definitive Guide to SQLite*, 2nd ed. New York, NY, USA: Apress, 2010.
- [25] R. T. Fielding a R. N. Taylor, „Principled design of the modern web architecture,“ *ACM Trans. Internet Technology*, vol. 2, no. 2, pp. 115–150, May 2002.
- [26] Meta Platforms, „React Native Documentation,“ *Meta*, 2024. [Online]. Dostupné: <https://reactnative.dev/docs/getting-started>.
- [27] Expo, „Expo SDK 54 Documentation,“ *Expo*, 2024. [Online]. Dostupné: <https://docs.expo.dev/>.
- [28] Expo, „Expo Router Documentation,“ *Expo*, 2024. [Online]. Dostupné: <https://docs.expo.dev/router/introduction/>.
- [29] Microsoft, „TypeScript 5.9 Handbook,“ 2024. [Online]. Dostupné: <https://www.typescriptlang.org/docs/handbook/intro.html>.
- [30] G. E. P. Box, G. M. Jenkins, G. C. Reinsel a G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.
- [31] T. Chen a C. Guestrin, „XGBoost: A scalable tree boosting system,“ in *Proc. 22nd ACM SIGKDD Int. Conf. KDD*, 2016, pp. 785–794.
- [32] S. Hochreiter a J. Schmidhuber, „Long short-term memory,“ *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [33] I. Goodfellow, Y. Bengio a A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [34] E. I. Vlahogianni, K. Kepaptsoglou, V. Tsetos a M. G. Karlaftis, „A real-time parking prediction system for smart cities,“ *J. Intell. Transp. Syst.*, vol. 20, no. 2, pp. 192–204, Mar. 2016.
- [35] K. J. Åström a R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, 2nd ed. Princeton, NJ, USA: Princeton Univ. Press, 2021.
- [36] N. Wiener, *Cybernetics, or Control and Communication in the Animal and the Machine*, 2nd ed. Cambridge, MA, USA: MIT Press, 1961.
- [37] T. Huang, G. Yang a G. Tang, „A fast two-dimensional median filtering algorithm,“ *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 27, no. 1, pp. 13–18, Feb. 1979.
- [38] S. Lloyd, „Programming the universe: A finite-state-machine view of physical computation,“ *Information Sciences*, vol. 175, no. 4, pp. 209–220, Nov. 2005.
- [39] E. F. Codd, „A relational model of data for large shared data banks,“ *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970.
- [40] P. van der Waerden, A. Borgers a H. Timmermans, „A model of public parking choice behavior,“ *Transp. Planning and Technology*, vol. 28, no. 4, pp. 281–301, Aug. 2005.
- [41] P. Horowitz a W. Hill, *The Art of Electronics*, 3rd ed. Cambridge, U.K.: Cambridge Univ. Press, 2015.
- [42] M. Fowler, „Event sourcing,“ *martinfowler.com*, 2005. [Online]. Dostupné: <https://martinfowler.com/eaDev/EventSourcing.html>.
- [43] J. Nielsen, *Usability Engineering*. San Francisco, CA, USA: Morgan Kaufmann, 1993.
- [44] T. Ali, M. Irfan, A. S. Alwadie a A. Glowacz, „IoT-based smart waste bin monitoring,“ *Arabian J. Sci. Eng.*, vol. 45, no. 12, pp. 10185–10198, Dec. 2020.
- [45] Z. Pala a N. Inanc, „Smart parking applications using RFID technology,“ in *Proc. 1st Annu. RFID Eurasia Conf.*, 2007, pp. 1–3.

- [46] M. Jones, J. Bradley a N. Sakimura, „JSON Web Token (JWT),“ IETF RFC 7519, May 2015. [Online]. Dostupné: <https://www.rfc-editor.org/rfc/rfc7519>.
- [47] ENISA, „Good Practices for Security of Internet of Things in the context of Smart Manufacturing,“ ENISA Report, Heraklion, Greece, Nov. 2018.
- [48] M. Centenaro, L. Vangelista, A. Zanella a M. Zorzi, „Long-range communications in unlicensed bands: The rising stars in the IoT and smart city scenarios,“ IEEE Wireless Commun., vol. 23, no. 5, pp. 60–67, Oct. 2016.
- [49] S. Sadowski a P. Spachos, „Wireless technologies for smart agricultural monitoring using IoT devices with energy harvesting capabilities,“ Computers and Electronics in Agriculture, vol. 172, pp. 1–9, May 2020.
- [50] R. Buyya, J. Broberg a A. M. Goscinski, Cloud Computing: Principles and Paradigms. Hoboken, NJ, USA: Wiley, 2011.